

Hoofdstuk 1: Javascript + Vite + Bootstrap projecttemplate.....	9
Hoofdstuk 2: Functions.....	12
Doel van dit hoofdstuk.....	12
Concept & uitleg.....	12
Voorbeeld in jouw template.....	13
Mini-opdracht.....	14
Begeleide oplossing.....	15
Hoofdstuk 3: Function Scoping.....	16
Concept & uitleg.....	16
Denkbeeld.....	17
Oefening: Function Scope Demo.....	17
Mini-opdracht.....	18
Begeleide oplossing.....	19
Wat je nu beheerst.....	19
Hoofdstuk 4: Named Functions.....	20
Theorie: Named Functions.....	20
Demo: Kleur tonen met named functions.....	20
Wat zagen we?.....	22
Mini-opdracht.....	22
Begeleide oplossing mini-opdracht.....	23
Samenvatting.....	24
Hoofdstuk 5: Functions & Strings in ES6 (Template Literals).....	24
Theorie: Template Literals.....	24
Real-life analogie.....	25
Demo: Welkomstbericht generator.....	25
Mini-opdracht.....	27
Mini-opdracht begeleide oplossing.....	27
Samenvatting.....	28
Hoofdstuk 6: ES6 Arrow Functions (=>).....	29
Theorie: Arrow Functions.....	29
Vergelijking.....	29
Let op.....	30
Oefening: Bereken + Toon (arrow style).....	30
Wat zag je?.....	31
Mini-opdracht.....	31
Oplossing mini-opdracht.....	32
Samenvatting.....	32

Hoofdstuk 7: ES6 Arrays	33
Theorie: Arrays.....	33
Vergelijking in real life.....	33
Demo: gastenlijst (array + UI).....	34
Mini-opdracht.....	35
Mini-opdracht oplossing.....	36
Samenvatting.....	37
Hoofdstuk 8: DOM Nodes & Root Nodes	37
Theorie	37
Denkbeeld	38
Demo: tekst zetten via root nodes.....	38
Wat gebeurde hier?.....	40
Mini-opdracht.....	40
Mini-opdracht oplossing.....	40
Samenvatting.....	41
Volgend hoofdstuk.....	41
Hoofdstuk 9: JavaScript Output Methodes.....	42
Theorie: Output Methodes.....	42
Demo oefening: 3 outputmethoden.....	43
Mini-opdracht.....	44
Mini-opdracht oplossing.....	45
Samenvatting.....	45
Hoofdstuk 10: document.write() en document.writeln().....	46
Theorie	46
Demo: Waarom je het niet gebruikt.....	47
Mini-opdracht.....	48
Mini-opdracht oplossing.....	48
Samenvatting.....	49
Hoofdstuk 11: innerHTML vs textContent vs innerText.....	49
Theorie	49
Veiligheid tip.....	50
Demo: drie output methoden	50
Mini-opdracht.....	52
Oplossing mini-opdracht.....	52
Samenvatting.....	53
Hoofdstuk 12: DOM Selectors.....	53
Theorie: DOM Selectors.....	53

Denkbeeld	54
Demo: lijst filteren met querySelectorAll.....	54
Mini-opdracht.....	55
Oplossing mini-opdracht.....	55
Samenvatting.....	56
Hoofdstuk 13: Andere DOM-element properties & acties.....	56
Theorie	56
Metafoor.....	57
Demo: focus highlight + child counter	57
Mini-opdracht.....	59
Mini-opdracht oplossing.....	59
Samenvatting.....	60
Hoofdstuk 14: Klassen manipuleren (classList)	61
Theorie	61
Demo: Highlight toggle + knop states.....	61
Mini-opdracht.....	63
Mini-opdracht oplossing.....	63
Samenvatting.....	64
Hoofdstuk 15: Events & Formulieren.....	64
Theorie: Wat is een Event?	64
Metafoor.....	65
Demo: Live naam weergave + form submit.....	65
Mini-opdracht.....	67
Mini-opdracht oplossing.....	67
Samenvatting.....	68
Hoofdstuk 16: Event Types & preventDefault() (Dieper) + Mouse Events.....	69
Theorie	69
Demo: Link + Box hover effect + Submit block.....	70
Mini-opdracht.....	71
Oplossing mini-opdracht.....	72
Samenvatting.....	72
Hoofdstuk 17: Mini-Project – Todo App (Events + Arrays + DOM).....	73
Uitleg.....	73
UI Design.....	73
index.html.....	74
main.js.....	74
Demo features die studenten leren.....	75

Mini-Opdracht.....	76
Samenvatting.....	76
Hoofdstuk 18: Objects & Date() in JavaScript.....	77
Theorie: Wat is een object?	77
Theorie: Date() object.....	77
Use case: Leeftijd berekenen.....	78
Demo: Leeftijd-calculator.....	78
Mini-opdracht.....	79
Mini-opdracht oplossing.....	80
Samenvatting.....	81
Hoofdstuk 19: Object Constructors & Inheritance (Intro).....	81
Theorie.....	82
Demo: User maken en tonen.....	82
Mini-opdracht.....	84
Oplossing mini-opdracht.....	84
Inheritance Light.....	85
Samenvatting.....	85
Hoofdstuk 20: ES6 Classes + extends + super.....	86
Theorie: Wat is een Class?.....	86
Inheritance met extends.....	87
Demo: User + Admin UI.....	87
Mini-opdracht.....	89
Samenvatting.....	89
Hoofdstuk 21: Mini Project – Product Catalog (Webshop Logica).....	90
Concept.....	90
Class Model.....	90
HTML UI.....	90
Styles (optioneel animatie).....	91
JavaScript.....	91
Test checklist.....	92
Mini-opdracht (uitbreiding).....	93
Samenvatting.....	93
Hoofdstuk 22: Browser Object Model (BOM)	94
Theorie.....	94
Veiligheid: popups & redirects.....	95
Demo: BOM Info Display.....	95
Mini-opdracht.....	96

Mini-opdracht oplossing.....	96
Samenvatting.....	97
Hoofdstuk 23: Cookies & LocalStorage.....	98
Demo: Naam onthouden.....	99
Demo 2: Array opslaan (favorietenlijst).....	100
Mini-opdracht.....	101
Samenvatting.....	102
Hoofdstuk 24: AJAX & API's Intro.....	103
Theorie.....	103
Voorbeeld van async denken.....	104
Demo: Data ophalen met fetch().....	104
Wat leerde je eigenlijk?.....	105
Mini-opdracht.....	105
Samenvatting.....	106
Hoofdstuk 25: JSON File Load + Local API Simulator.....	106
Wat is JSON?.....	106
JSON in Vite project plaatsen.....	107
UI: Lokaal JSON bestand laden.....	107
UI resultaat.....	108
Mini-opdracht.....	109
Mini-opdracht oplossing (docentversie).....	109
Samenvatting.....	110
Hoofdstuk 26: POST met Fetch (Fake API) & CRUD Intro.....	111
Theorie.....	111
Denk in stappen.....	111
Mini Fake API: Create Product.....	111
index.html.....	112
main.js.....	112
Wat werkt hier?.....	113
Mini-Opdracht.....	113
Mini-Opdracht Oplossing (docentversie).....	114
Samenvatting.....	114
Hoofdstuk 27: PUT & DELETE (Fake CRUD).....	115
Theorie.....	115
Mini Fake Admin Panel.....	115
index.html (blok toevoegen).....	115
main.js.....	116

Mini-opdracht.....	118
Samenvatting.....	118
Hoofdstuk 28: Echte HTTP Requests (fetch POST/PUT/DELETE) + API Testing Tools.....	119
Theorie.....	119
API Tools uitleg.....	119
Belangrijk.....	120
Demo 1: GET request (opfrisser).....	120
Demo 2: POST request (echt).....	120
Demo 3: DELETE request (echt).....	121
Wat studenten nu snappen.....	122
Mini-opdracht.....	122
Samenvatting.....	122
Hoofdstuk 29: Async/Await Deep Dive + UX voor Loading & Errors.....	123
Theory: Async/Await.....	123
UX Rules (professioneel).....	123
Demo: Fake API + Loading UI.....	124
Bonus: Retry button.....	125
Advanced Concepts (kort).....	125
Mini-opdracht.....	126
Mini-opdracht Oplossing (docentversie, samenvatting).....	126
Samenvatting.....	126
Hoofdstuk 30: Fetch Timeout, AbortController & Race Conditions.....	127
Theorie.....	127
UX Patterns.....	127
Demo: Cancel & Timeout.....	128
Wat werkt hier?.....	129
Race Condition Uitleg.....	129
Mini-opdracht.....	130
Docent-oplossing (verkort).....	130
Samenvatting.....	130
Hoofdstuk 31: Multiple Requests + Promise.all + Loading Skeletons.....	131
Theorie.....	131
Skeleton UI.....	131
Skeleton CSS.....	131
Demo: Parallel API Load (users + posts).....	132
UX Resultaat.....	133
Mini-opdracht.....	134

Samenvatting.....	134
Hoofdstuk 32: Modules, Imports & Professionele Projectstructuur.....	135
Waarom modules?.....	135
Nieuwe Project structuur.....	136
We bouwen: User Card Component + API Service.....	136
js/components/userCard.js.....	136
js/utils/format.js.....	136
js/main.js.....	137
index.html voor test.....	137
Wat je nu hebt gebouwd.....	137
Extra inzichten.....	137
Mini-opdracht.....	138
Docent-oplossing hints.....	138
Samenvatting.....	139
Hoofdstuk 33: Eindproject — Mini Shopping Dashboard.....	139
Project Concept.....	139
Structuur.....	140
data/products.json.....	140
Utilities.....	140
Components.....	141
Services.....	141
HTML sectie.....	142
js/main.js.....	142
Wat dit project demonstreert.....	143
Mini-opdracht (student).....	144
Wat studenten nu kunnen.....	144
Hoofdstuk 34: Developer Tools & Debugging Mastery.....	145
Theorie (kort & scherp).....	145
Demo 1 — Console power: table, group, fouten lezen.....	145
Demo 2 — Breakpoints, Watch & Call Stack.....	146
Demo 3 — Network: Throttling & Offline, Errors lezen.....	147
Mini-opdracht.....	149
Begeleide oplossing (met inline comments).....	149
Samenvatting.....	150
Hoofdstuk 35: ES6+ Advanced Patterns & Array Mastery.....	151
Theorie.....	151
Demo 1 — Destructuring + Spread + Rest.....	152

Demo 2 — map, filter, reduce.....	153
Immutability (zoals React).....	154
Mini-opdracht.....	154
Samenvatting.....	155
Hoofdstuk 36: Modern Forms + Validatie + UX Patterns.....	155
Theorie.....	155
Demo Form: Login + Live Feedback.....	156
Mini-Opdracht.....	157
Oplossing (docentversie).....	158
Samenvatting.....	158

Hoofdstuk 1: Javascript + Vite + Bootstrap projecttemplate

Doel van dit hoofdstuk

In dit hoofdstuk maken we een basisproject dat we **bij elke oefening** opnieuw gebruiken. Je leert:

- een Vite projectstructuur begrijpen
- Bootstrap via npm + SCSS integreren
- een testcomponent bouwen om te controleren of alles werkt
- een vaste oefen-template inzetten voor alle volgende hoofdstukken

Na dit hoofdstuk heb je een solide startpunt voor alle volgende JavaScript-oefeningen.

Concept en uitleg

Waarom Vite?

Vite is een moderne bundler met:

- bliksemsnelle development server
- native ES modules (super logisch voor modern JS)
- automatische refresh bij codewijzigingen
- simpele build-flow voor productie

Waarom Bootstrap via npm?

We gebruiken npm zodat we:

- Bootstrap als onderdeel van onze build hebben
- SCSS kunnen overschrijven en uitbreiden
- géén CDN nodig hebben
- professioneel en future-proof werken

Hoe gaan we oefenen?

Elke oefening start van dezelfde template:

- `index.html` voor markup
- `main.js` voor logica
- `styles.scss` voor styling
- Bootstrap layout voor oefen-UI

We verversen dus enkel de inhoud van de `main.js` en eventueel HTML-blokken. Dit geeft structuur en voorkomt chaos.

Template voorbeeld (Smoke Test)

Dit is het stukje dat **bewijst dat je setup werkt**. Als de knop werkt, ben je klaar voor de rest van de cursus. Je krijgt eerst de template van de docent.

Bestand: src/index.html

```
<div class="container py-4">
  <h1 class="h4 mb-3">Vite + Bootstrap werkt </h1>

  <div class="card shadow-sm">
    <div class="card-header bg-primary text-white">
      Test
    </div>
    <div class="card-body">
      <p class="mb-3">Klik op de knop om te testen of alles goed geladen
is.</p>

      <button id="btnTest" class="btn btn-primary">
        Test template
      </button>

      <div id="out" class="alert alert-secondary mt-3 mb-0">
        Output verschijnt hier
      </div>
    </div>
  </div>
</div>
```

Bestand: src/js/main.js

```
import './scss/styles.scss'
import * as bootstrap from 'bootstrap'

document.addEventListener('DOMContentLoaded', () => {
  const btn = document.getElementById('btnTest')
  const out = document.getElementById('out')

  btn.addEventListener('click', () => {
    out.className = 'alert alert-success mt-3 mb-0'
    out.textContent = ' Alles werkt. Klaar voor de oefeningen!'
  })
})
```

Bestand: src/scss/styles.scss

```
@import "bootstrap/scss/bootstrap";

body {
  background-color: #f8f9fa;
}
```

Als je nu `npm run start` doet en klikt op de knop, zie je een groene alert:

Alles werkt. Klaar voor de oefeningen!

Dan is je omgeving 100% klaar.

Mini-opdracht

Voeg onder de testcard een tweede card toe met:

- een tekstinput
- een knop "Toon naam"
- een alert die "Welkom, [naam]" toont

Gebruik Bootstrap classes.

Begeleide oplossing

Toevoeging in index.html (onder de testcard)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Naam test</div>
  <div class="card-body">
    <input id="inpName" class="form-control mb-2" placeholder="Geef je naam in" />
    <button id="btnName" class="btn btn-dark w-100">Toon naam</button>

    <div id="nameOut" class="alert alert-secondary mt-3 mb-0">Nog niets
ingegeven</div>
  </div>
</div>
```

Toevoeging in main.js

```
const btnName = document.getElementById('btnName')
const inpName = document.getElementById('inpName')
const nameOut = document.getElementById('nameOut')

btnName.addEventListener('click', () => {
  const naam = inpName.value.trim()

  if (!naam) {
    nameOut.className = 'alert alert-warning mt-3 mb-0'
    nameOut.textContent = '⚠️ Vul je naam in aub'
    return
  }

  nameOut.className = 'alert alert-success mt-3 mb-0'
  nameOut.textContent = `Welkom, ${naam}!`
})
```

Resultaat

Je hebt nu:

- een werkende Vite + Bootstrap basis
- een vaste oefentemplate
- UI-interactie met DOM
- duidelijke output feedback

We gaan vanaf nu **elke oefening in deze template schrijven**.

Hoofdstuk 2: Functions

Doel van dit hoofdstuk

Na dit hoofdstuk kan je:

- uitleggen wat een functie is
- een functie definiëren en aanroepen
- werken met parameters & return
- pure functies (berekening) scheiden van impure functies (UI)
- input lezen en output tonen met JS
- Bootstrap UI koppelen aan functies

Concept & uitleg

Wat is een functie?

Een functie is **herbruikbare code** die iets doet en (meestal) een resultaat teruggeeft.

Waarom?

- Nooit dubbel werk (**DRY** principe)
- Logica netjes gescheiden
- Helder, leesbaar, uitbreidbaar
- In UI: berekening apart, tonen apart

Structuur

```
function naam(parameter) {  
  return resultaat;  
}
```

naam(waarde)

Belangrijk

Term	Betekenis
Parameter	naam in functie definitie
Argument	waarde die je doorgeeft
Return	wat de functie teruggeeft

Pure vs Impure

Type	Doel
Pure	berekenen, niets aanpassen buiten functie
⚠ Impure	DOM, alert, console.log, data wijzigen

We gebruiken **pure functies** voor **logica**, en **impure functies** voor **UI**.

Voorbeeld in jouw template

We voegen **1 oefenblok** toe met unieke ID's.

src/index.html → onder de vorige testcard voeg je toe:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">Hoofdstuk 2: Functions</div>
  <div class="card-body">

    <label class="form-label">Wat is je naam?</label>
    <input id="fn_nameInput" class="form-control mb-2" placeholder="vb. Tom">

    <button id="fn_btnGreet" class="btn btn-primary w-100">Groet</button>

    <div id="fn_output" class="alert alert-secondary mt-3 mb-0">
      Resultaat komt hier...
    </div>

  </div>
</div>
```

Opgelet: **fn_ prefix** = “functions chapter”. Zo vermijden we ID-conflicten door heel de cursus.

src/js/main.js → voeg onderaan toe:

```
// -----
// Hoofdstuk 2: Functions
// -----

// Pure function (berekening)
function maakGroet(naam) {
  return `Hallo ${naam}!`;
}

// Impure function (UI)
function handleGroetClick() {
  const naamInput = document.getElementById('fn_nameInput');
  const output = document.getElementById('fn_output');

  const naam = naamInput.value.trim();

  if (!naam) {
    output.className = "alert alert-warning mt-3 mb-0";
    output.textContent = "⚠ Geef een naam in!";
  }
}
```

```

    return;
  }

  const boodschap = maakGroet(naam);

  output.className = "alert alert-success mt-3 mb-0";
  output.textContent = boodschap;
}

// Event koppelen
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById('fn_btnGreet')
    ?.addEventListener("click", handleGroetClick);
});

```

Expected behaviour

- Lege input → gele waarschuwing
- Geldige naam → groene groet

Dit lijkt simpel, maar vormt het fundament voor **ELK volgend hoofdstuk**.

Mini-opdracht

Maak functie berekenVierkant(getal):

Input	Output
5	"5 ² = 25"
10	"10 ² = 100"

UI:

- input veld (nummer)
- knop "Bereken vierkant"
- Bootstrap alert onderaan

Zelfde ID-stijl:

```

sq_input
sq_btn
sq_output

```

Begeleide oplossing

HTML (onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht: Vierkant
  berekenen</div>
  <div class="card-body">

    <input id="sq_input" type="number" class="form-control mb-2"
placeholder="vb. 5">
    <button id="sq_btn" class="btn btn-dark w-100">Bereken vierkant</button>

    <div id="sq_output" class="alert alert-secondary mt-3 mb-0">
      Resultaat komt hier...
    </div>

  </div>
</div>
```

JS

```
// Pure berekening
function berekenVierkant(getal) {
  return getal * getal;
}

// UI handler
function handleVierkant() {
  const inp = document.getElementById('sq_input');
  const out = document.getElementById('sq_output');

  const waarde = Number(inp.value);

  if (!waarde) {
    out.className = "alert alert-warning mt-3 mb-0";
    out.textContent = "⚠ Geef een getal in!";
    return;
  }

  const resultaat = berekenVierkant(waarde);

  out.className = "alert alert-success mt-3 mb-0";
  out.textContent = `${waarde}2 = ${resultaat}`;
}

// Event koppelen
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById('sq_btn')
    ?.addEventListener("click", handleVierkant);
});
```

Hoofdstuk 3: Function Scoping

Doel van dit hoofdstuk

Na dit hoofdstuk kan je:

- uitleggen wat **scope** betekent
- het verschil begrijpen tussen **function scope** en **block scope**
- lokale vs globale variabelen herkennen
- nested functions gebruiken
- scoping-fouten herkennen zoals `ReferenceError: x is not defined`

Concept & uitleg

Wat is scope?

Scope bepaalt **waar** een variabele toegankelijk is.

Scope type	Betekenis
Global scope	overal zichtbaar
Function scope	enkel binnen een functie zichtbaar
Block scope (<code>let</code> , <code>const</code>)	enkel binnen <code>{ }</code> zichtbaar
Lexical scope	inner functions kunnen outer data gebruiken

ES6 sleutelregel

Gebruik `let` en `const` ✗ gebruik geen `var` meer ondanks dat dit altijd zal blijven werken (oude function-scoped variabelen)

Klassieke bug

```
{  
  let x = 5;  
}  
console.log(x); // ✗ ReferenceError
```

Nested functions

```
function outer() {  
  const naam = "Tom";  
  
  function inner() {  
    console.log(naam); // werkt  
  }  
  
  inner();  
}
```

Binnenste ziet buitenste. Buitenste ziet binnenste **niet**.

Denkbeeld

Stel je voor:

- Je code is een huis.
- Een functie is een **kamer**.
- Wat in de kamer ligt, kan je daar gebruiken.
- Maar je kan niet door de muur kijken naar binnen.

Inner functions = een klein kamertje in de kamer.

Oefening: Function Scope Demo

src/index.html → voeg dit oefenblok toe:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">Hoofdstuk 3: Function
  Scoping</div>
  <div class="card-body">

    <button id="sc_btn" class="btn btn-info w-100 mb-2">Voer scope demo
    uit</button>

    <div id="sc_output" class="alert alert-secondary mb-0">
      Klik op de knop om scope te testen...
    </div>

  </div>
</div>
```

src/js/main.js → voeg onderaan toe:

```
// -----
// Hoofdstuk 3: Function Scoping
// -----

function toonBericht() {
  const binnen = "Ik leef binnen de functie 🐼";

  function inner() {
    return `Inner ziet: ${binnen}`;
  }

  return inner();
}

function handleScope() {
  const output = document.getElementById("sc_output");

  // inner sees outer
  const bericht = toonBericht();

  output.className = "alert alert-success mb-0";
  output.textContent = bericht;
}
```

```

try {
  // ✗ buiten de functie proberen de variabele te gebruiken
  console.log(binnen);
} catch (err) {
  console.warn("binnen is niet zichtbaar buiten de functie");
}
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("sc_btn")
    ?.addEventListener("click", handleScope);
});

```

Expected behaviour

Klik op de knop → groene alert:

Inner ziet: Ik leef binnen de functie 🗝️

Console:

binnen is niet zichtbaar buiten de functie

Mini-opdracht

Maak:

- functie geheimBericht()
- lokale variabele secret = "Code unlocked 🗝️"
- nested function leesSecret()
- return dat inner result
- UI met knop + output

Gebruik ID's:

sc2_btn
sc2_output

Hint: zelfde patroon als boven, andere tekst.

Begeleide oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-warning">Mini-opdracht: Geheim bericht</div>
  <div class="card-body">
    <button id="sc2_btn" class="btn btn-warning w-100 mb-2">Toon
geheim</button>
    <div id="sc2_output" class="alert alert-secondary mb-0">
      Wacht op geheim...
    </div>
  </div>
</div>
```

JS

```
function geheimBericht() {
  const secret = "Code unlocked 🔓";

  function leesSecret() {
    return secret;
  }

  return leesSecret();
}

function handleSecret() {
  const out = document.getElementById("sc2_output");
  out.className = "alert alert-success mb-0";
  out.textContent = geheimBericht();
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("sc2_btn")
    ?.addEventListener("click", handleSecret);
});
```

Wat je nu beheerst

- Function scope
- Block scope
- Lexical scope
- Inner vs outer toegang
- Variable accessibility errors herkennen

Dit is superbelangrijk voor:

- Callbacks
- Modules
- DOM events
- Promises / async
- API calls later

Hoofdstuk 4: Named Functions

Doel van dit hoofdstuk

Na deze module kan je:

- uitleggen wat een **named function** is
- function declarations schrijven
- named functions koppelen aan knoppen
- het verschil begrijpen tussen **pure** en **impure** functies
- foutmeldingen beter lezen dankzij functienamen
- input → verwerking → output met duidelijke code

Theorie: Named Functions

Wat is een named function?

Een functie met een **duidelijke naam**:

```
function zegHallo() {  
  return "Hallo!";  
}
```

Waarom belangrijk?

Reden	Waarom
Lesbaarheid	Je ziet in één oogopslag wat code doet
Debugging	Foutmeldingen tonen functienaam
Structuur	Bouwblokken die makkelijk herbruikbaar zijn
Teamwerk	Iedereen begrijpt het sneller

Named vs Anonymous

Slecht (moeilijk debuggen)

```
button.addEventListener("click", function() {  
  ...  
});
```

Goed (duidelijk & traceerbaar)

```
button.addEventListener("click", toonKleur);
```

Demo: Kleur tonen met named functions

We bouwen:

- inputveld
- knop
- tekstvak
- gekleurde box (visuele feedback)

src/index.html (+ blok onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-secondary text-white">
    Hoofdstuk 4: Named Functions
  </div>
  <div class="card-body">

    <label class="form-label">Typ een primaire kleur (rood of blauw):</label>
    <input id="nf_input" class="form-control mb-2" placeholder="bv. rood">

    <button id="nf_btn" class="btn btn-secondary w-100 mb-2">Toon
kleur</button>

    <div id="nf_text" class="alert alert-secondary mb-2">
      Wacht op input...
    </div>

    <div id="nf_box"
      class="d-flex justify-content-center align-items-center fw-bold"
      style="height: 70px; border-radius: 6px; border: 1px solid #ccc;
background:#f8f9fa;">
      Geen kleur
    </div>

  </div>
</div>
```

src/js/main.js (+ onderaan toevoegen)

```
// -----
// Hoofdstuk 4: Named Functions
// -----

// Pure function
function bepaalKleurInfo(kleur) {
  const input = kleur.toLowerCase();

  if (input === "rood") return { text: "Je koos rood", color: "red" };
  if (input === "blauw") return { text: "Je koos blauw", color: "blue" };

  return { text: "Onbekende kleur", color: null };
}

// Impure function (DOM)
function toonKleur() {
  const inp = document.getElementById("nf_input");
  const out = document.getElementById("nf_text");
  const box = document.getElementById("nf_box");

  const waarde = inp.value.trim();

  if (!waarde) {
    out.className = "alert alert-warning mb-2";
    out.textContent = "⚠ Geef een kleur in";
  }
}
```

```

    box.style.background = "#f8f9fa";
    box.textContent = "Geen kleur";
    return;
}

const resultaat = bepaalKleurInfo(waarde);

if (!resultaat.color) {
    out.className = "alert alert-danger mb-2";
    out.textContent = resultaat.text;
    box.style.background = "#f8f9fa";
    box.textContent = "Onbekend";
    return;
}

out.className = "alert alert-success mb-2";
out.textContent = resultaat.text;
box.style.background = resultaat.color;
box.textContent = resultaat.color.toUpperCase();
}

// Event koppeling
document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("nf_btn")
        ?.addEventListener("click", toonKleur);
});

```

Wat zagen we?

- Functie **met naam** is beter traceerbaar
- Pure functie berekent
- Impure functie werkt met UI
- Input → verwerking → output + visuele feedback

Mini-opdracht

Maak een **fruit-kiezer** met emoji.

Vereisten

Input	Tekst	Emoji
appel	"Je koos appel"	🍏
banaan	"Je koos banana"	🍌
anders	waarschuwing	?

UI-elementen

- Input veld
- Knop
- Tekstvak (alert)
- Blok met emoji

Verplichte ID's

fr_input
fr_btn
fr_text
fr_box

Tip

Pure function retourneert:

```
{ text: "Je koos appel", emoji: "🍏" }
```

Begeleide oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-warning">Mini-opdracht: Fruit kiezer</div>  
  <div class="card-body">  
  
    <input id="fr_input" class="form-control mb-2" placeholder="bv. appel">  
    <button id="fr_btn" class="btn btn-warning w-100 mb-2">Kies  
fruit</button>  
  
    <div id="fr_text" class="alert alert-secondary mb-2">Wacht op  
input...</div>  
  
    <div id="fr_box"  
      class="d-flex justify-content-center align-items-center fw-bold"  
      style="height:70px; border:1px solid #ccc; border-radius:6px;  
background:#f8f9fa;">  
      ?  
    </div>  
  
  </div>  
</div>
```

JS

```
function bepaalFruit(naam) {  
  const f = naam.toLowerCase();  
  
  if (f === "appel") return { text: "Je koos appel", emoji: "🍏" };  
  if (f === "banaan") return { text: "Je koos banaan", emoji: "🍌" };  
  
  return { text: "Onbekend fruit", emoji: "?" };  
}  
  
function toonFruit() {  
  const inp = document.getElementById("fr_input");  
  const txt = document.getElementById("fr_text");  
  const box = document.getElementById("fr_box");  
  
  const waarde = inp.value.trim();
```

```

if (!waarde) {
  txt.className = "alert alert-warning mb-2";
  txt.textContent = "⚠️ Vul een fruitsoort in!";
  box.textContent = "?";
  return;
}

const res = bepaalFruit(waarde);

txt.className = "alert alert-success mb-2";
txt.textContent = res.text;
box.textContent = res.emoji;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("fr_btn")
    ?.addEventListener("click", toonFruit);
});

```

Samenvatting

Concept	Belangrijk
Named functions	leesbaar, debugbaar
Pure functions	berekenen, géén DOM
Impure	DOM manipulatie
Mapping	user input naar UI resultaat
UX	visuele feedback is cruciaal

Hoofdstuk 5: Functions & Strings in ES6 (Template Literals)

Leerdoelen

Na dit hoofdstuk kan je:

- ES6 **template literals** gebruiken (``)`)
- variabelen en functies verwerken in strings
- multiline strings schrijven
- dynamische tekst genereren in UI
- string-interpolatie gebruiken in JS functies

Theorie: Template Literals

Oude manier (slecht leesbaar)

```

const naam = "Tom";
const zin = "Hallo, " + naam + "! Welkom.";

```

Moderne ES6 manier

```

const naam = "Tom";
const zin = `Hallo, ${naam}! Welkom.`;

```

Voordelen

Voordeel	Waarom
Leesbaar	natuurlijke zinnen
Variabelen in string	<code>\${variabele}</code>
Multiline strings	geen <code>\n</code> nodig
Perfect voor HTML	dynamische UI content

Multiline voorbeeld

```
const blokje = `
Dit is een titel
En dit is een tweede lijn
Het leest heerlijk
`;
```

Real-life analogie

Template literals zijn **post-its met invulvelden**:

“Welkom, **{naam}**! Je score is **{score}**.”

In JS:

```
`${naam} ${score}`
```

Demo: Welkomstbericht generator

We bouwen:

- input veld voor naam
- input veld voor leeftijd
- knop
- output met template literal

index.html (plaats onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">
    Hoofdstuk 5: Template Literals
  </div>
  <div class="card-body">

    <label class="form-label">Naam:</label>
    <input id="tl_name" class="form-control mb-2" placeholder="bv. Tom">

    <label class="form-label">Leeftijd:</label>
    <input id="tl_age" type="number" class="form-control mb-2"
placeholder="bv. 25">

    <button id="tl_btn" class="btn btn-info w-100 mb-2">Maak
welkomstbericht</button>

    <div id="tl_output" class="alert alert-secondary mb-0">
      Wacht op input...
```

```

    </div>

</div>
</div>

main.js (onderaan toevoegen)
// -----
// Hoofdstuk 5: Template Literals
// -----

// Pure function
function maakWelkomstZin(naam, leeftijd) {
  return `Welkom ${naam}! Jij bent ${leeftijd} jaar oud.`;
}

// UI handler
function toonWelkomstZin() {
  const naam = document.getElementById("tl_name").value.trim();
  const leeftijd = document.getElementById("tl_age").value.trim();
  const out = document.getElementById("tl_output");

  if (!naam || !leeftijd) {
    out.className = "alert alert-warning mb-0";
    out.textContent = `⚠️ Vul naam en leeftijd in`;
    return;
  }

  const tekst = maakWelkomstZin(naam, leeftijd);

  out.className = "alert alert-success mb-0";
  out.textContent = tekst;
}

// Event
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("tl_btn")
    ?.addEventListener("click", toonWelkomstZin);
});

```

Output type:

Welkom Tom! Jij bent 25 jaar oud.